“What’s That Book Called?”

A Personal AI-Powered Book Finder for Fuzzy Memory Search

1. Introduction

For this final project, I propose building a personal AI-powered book finder that helps organize and retrieve books based on vague or incomplete memories. The goal is to create a tool that acts like a searchable memory, helping users rediscover books even when they cannot recall precise details. By using AI-based semantic search, the system can match fuzzy descriptions to the most relevant books in the user’s personal library. The purpose of this project is to explore how AI techniques learned in class can be applied to a practical, real-life problem while building a tool that is genuinely useful for personal use.

2. Problem Background

As someone who reads frequently, I often read books from various online platforms rather than from a single source. Over time, this has made it increasingly difficult to keep track of all the books I have read, especially since they are spread across different websites and applications. Unlike physical books, digital reading does not provide a clear or permanent record of ownership, which makes revisiting a book much harder.

A problem I personally encounter is wanting to reread a book but being unable to remember its title or the platform where I originally read it. In many cases, I only remember vague details such as a general storyline, a character’s personality, or a specific emotional moment. When this happens, traditional search methods are usually ineffective because they require exact keywords, titles, or author names, none of which I can recall accurately.

This experience highlights the mismatch between how I remember stories and how conventional search systems work. My memory of a book is often incomplete and based on impressions rather than structured information. Because of this, there is a need for a system that can work with vague, natural language descriptions and still return meaningful results. This project is motivated by that personal frustration and aims to explore how AI-based semantic search can better support the way humans naturally remember and search for information.

3. Solution Overview

The system is implemented as a simple interactive Python program designed to be easy to test and extend. Instead of building a complex interface, I focused on making the logic clear and functional.

- Book Storage and Data Structure
  - Books are stored as a list of dictionaries. Each book contains:
  - title
  - short summary (written based on my memory)
  - tags
  - rating
  - reading link
  - revisit count

  All book data is saved in a JSON file so it persists across sessions. Autosave is enabled, meaning any changes are saved immediately without requiring manual actions.

- Embedding Generation
  When a book is added, the system generates an embedding using the book's summary and tags. These embeddings represent the semantic meaning of each book and are stored together with the book data. This allows the system to compare books based on meaning rather than exact words.
- Search Mechanism
  When searching, the user inputs a vague natural language description. The input is converted into an embedding and compared against all stored book embeddings using cosine similarity. Books with higher similarity scores are ranked higher and shown to the user.
- Interactive Refinement
  If the search results are unclear, the system does not stop immediately. Instead, it asks the user for more details and repeats the search using the refined description. This makes the search process more flexible and closer to how humans recall information.
- Personalization
  The system tracks how often a book is revisited. Books that are opened more frequently receive a small ranking boost during future searches. This allows the system to adapt to personal reading preferences over time.

4. Technologies Used

- Python
  Used as the main programming language to implement the entire system logic.
- Google Colab
  Used as the development and execution environment for testing and running the program.

- Sentence Transformer (Embedding Model)
  Used to convert book summaries, tags, and search queries into vector embeddings that capture semantic meaning.
- NumPy
  Used for numerical operations, including vector handling and cosine similarity computation.
- Cosine Similarity
  Used to measure semantic similarity between the search query and stored books.
- JSON
  Used to store the book library persistently, allowing data to be saved and loaded across sessions.

5. Implementation and Thought Process

At the beginning of this project, my implementation was very simple and limited. I initially focused on storing books in a basic list and matching search queries using straightforward logic. This approach worked only when the search description closely matched the stored summaries, and it quickly became clear that this was not enough to handle vague memories. I also underestimated how often small design decisions, such as how data was saved or when search results should stop, would affect the overall usability of the system.

During development, I revised the code multiple times after testing and realizing its weaknesses. For example, the search would sometimes stop too early without helping me narrow down the results, which felt frustrating as a user. I had to rethink how the search flow should behave and added an interactive refinement step so the system would ask for more details instead of giving up. I also encountered issues with data persistence and learned that relying on manual saving was error-prone, which led me to redesign the system with automatic saving to ensure that added books and revisit counts were not accidentally lost.

By the final stage, the system became more stable and closer to my original goal. The search process was able to handle vague descriptions more naturally, and the refinement step made it feel more like how I actually recall books over time. Adding the revisit count feature also helped personalize the results based on my own reading behavior. Through these iterations, I learned that building an AI system is not just about choosing a model, but also about testing, failing, and repeatedly improving the surrounding logic to create a better user experience.

6. Final Results

After testing the system with multiple vague and informal descriptions, the search was able to return relevant books without requiring exact titles or keywords. Queries based on general plot ideas, character relationships, or common tropes consistently produced reasonable results, showing that the embedding-based search could match descriptions by meaning rather than wording.

The revisit count feature worked as intended, with frequently selected books gradually appearing higher in later searches. When search results were unclear or too similar, the system successfully prompted for additional details and refined the search instead of stopping immediately. Overall, the testing confirmed that the system functions as expected and can effectively retrieve books based on fuzzy memory descriptions.

Coding Project:
https://colab.research.google.com/drive/1hMC1PX0OR0NOpbeAjBV8amq_eya0-C4H?usp=sharing

7. Reflection

Through this project, I gained a clearer understanding of how AI techniques can be applied to real-life problems in a practical way. One important realization I had was that the effectiveness of this system is closely related to the size and diversity of the data it works with. With a relatively small personal library, the system was able to perform reasonably well and return meaningful results based on vague descriptions. However, I also recognized that if I were to include all the books I have read over the past six years, the current implementation might no longer be sufficient and would require more advanced methods to maintain accuracy and efficiency.

During the development process, I encountered several moments where the system did not behave as expected, which forced me to rethink and revise the code multiple times. These adjustments helped me realize that building an AI-based system is rarely straightforward. Even small design choices, such as how search results are refined or how user interactions are handled, can significantly affect the overall experience. This iterative process taught me that improvement often comes from testing, failing, and refining rather than getting everything right on the first attempt.

Overall, this project changed how I view AI development. Instead of focusing only on the model itself, I learned to consider the surrounding system design, user behavior, and long-term scalability. Although the current version of the project is relatively simple, it successfully demonstrates the potential of AI-based semantic search. In the future, I would like to enhance this project by improving scalability, optimizing search performance, and possibly extending it into a more complete personal reading management tool that can support a much larger collection.

8. Conclusion

In this project, I built a personal AI-powered book finder that allows books to be retrieved using vague, natural language descriptions instead of exact titles. By applying embedding-based semantic search and interactive refinement, the system is able to match fuzzy memories to relevant books in a practical way. This project demonstrates how AI techniques learned in class can be used to solve a real-life problem and highlights the value of combining AI models with thoughtful system design to create useful applications.

9. Code and Resources

I used ChatGPT as a supporting tool during this project for brainstorming ideas, clarifying concepts, and helping with debugging and problem-solving when I encountered issues. ChatGPT was used as a reference and learning aid, similar to consulting documentation or online forums.

All implementation, testing, revisions, and final decisions were done by me. The project logic, structure, and final results reflect my own work and understanding, with ChatGPT providing guidance rather than complete solutions.